PIMPRI CHINCHWAD COLLEGE OF ENGINEERING, PUNE

Department of Information Technology
Design and Algorithm Laboratory

Assignment No. 4

Smart Traffic Management for Emergency Vehicles
(Dijkstra's Algorithm)

Name	Mohit Sojal
PRN	124B1F027
Class / Division	TY IT
Batch	A2
Subject	Design and Algorithm Laboratory
Faculty In-charge	Prof. Rajesh Phursule

1. Problem Statement

A smart city wants its traffic management system to get an ambulance to a hospital as fast as possible. The city's road network is modelled as a weighted, undirected graph: each intersection is a node, and each road connecting two intersections is an edge whose weight is the road's current travel time in minutes, which rises and falls with congestion. An ambulance starts at a known source intersection S and must be directed to whichever hospital among several candidate destinations can be reached soonest.

As the transportation engineer responsible for the routing logic, the assignment covers:

- Implementing Dijkstra's algorithm to compute the shortest travel time from the ambulance's location S to every intersection, so the nearest of several candidate hospitals can be identified.
- Recomputing the route whenever live traffic data changes a road's weight, without redesigning the algorithm.
- Keeping the solution efficient enough for a city-scale network of thousands of intersections and roads.
- Recovering and displaying the actual turn-by-turn path to the chosen hospital, not just its travel time, so it can be used for navigation.

Expected outcome: given the ambulance's current location and a set of hospitals, the system reports the nearest reachable hospital, its travel time, and the exact route to follow, and updates all three the moment new congestion data arrives.

2. Course Objective

- To build an understanding of how the running time and memory requirements of an algorithm grow with input size.
- To identify and apply an appropriate algorithmic strategy - such as Divide and Conquer, Greedy, or Dynamic Programming - while solving practical problems.

3. Course Outcome

- Analyse the asymptotic performance of a greedy shortest-path algorithm on a large graph.
- Apply the Greedy paradigm to design a correct and efficient solution to a real-time routing problem.

4. Theory

Dijkstra's algorithm computes the shortest travel time from one source intersection to every other intersection in a road network, provided no road has a negative travel time. It grows a set of "finalized" intersections one at a time: at each step it locks in whichever unfinalized intersection currently has the smallest known travel time, and uses that newly confirmed value to see whether it opens up a shorter route to any of its neighbours.

4.1 How the Algorithm Proceeds

- Initialization: the ambulance's starting intersection is given a travel time of 0, and a parent of “none”; every other intersection starts at travel time infinity with no parent, since no route has been found yet.
- Greedy selection: of all intersections not yet finalized, the one with the smallest currently known travel time is extracted next - a binary-heap min-priority queue makes this extraction efficient.
- Relaxation: for every road leaving the extracted intersection, the algorithm checks whether routing through it beats the neighbour's current best time; if so, the neighbour's distance is lowered and its parent is set to the extracted intersection, which is what lets the final route be rebuilt later.
- Termination: once every reachable intersection has been extracted and relaxed, every distance - and every parent link - is final.

4.2 Why the Greedy Choice Works Here

- Greedy-choice property: since no travel time is negative, the intersection with the smallest tentative distance can never later be undercut by a longer detour, so it is always safe to finalize the moment it is picked.
- Optimal substructure: the shortest route to any intersection v that passes through intersection u necessarily contains the shortest route to u, which is exactly why distances can be built up incrementally instead of re-deriving whole routes from scratch.

4.3 Handling Dynamic Traffic Conditions

Congestion is never static - it builds up, eases, or shifts across the day, changing how long a given road segment actually takes to drive. Rather than treating the graph as fixed, this assignment applies incoming live travel-time updates directly to the affected road segments and simply re-runs Dijkstra's algorithm from the ambulance's current location on the updated graph. Because each run is independent and costs only O((V + E) log V), recomputing the route after a traffic update is cheap enough to do in real time, so the dispatch system can react as conditions change instead of committing to a route that may no longer be the fastest one available.

4.4 Time and Space Complexity

Aspect	Complexity
Using a binary-heap priority queue	O((V + E) log V)
Using an adjacency-list graph representation	O(V + E) space
Re-routing after a traffic update	Same O((V + E) log V) bound, re-run per update
Reconstructing the path to a hospital	O(path length), via parent[]

Here V is the number of intersections and E is the number of roads in the city's network. The adjacency-list representation keeps memory proportional to the actual size of the road network rather than to the square of the number of intersections, which is what makes the approach viable for a city with thousands of junctions.

5. Algorithm

- Set the ambulance's starting intersection to a distance of 0 with no parent; set every other intersection to infinity; insert the source into a min-priority queue.
- While the queue is not empty, remove the intersection u with the smallest distance recorded in the queue.
- If the removed distance is greater than the best distance already known for u, discard it as a stale entry and continue.
- Otherwise, for every road connecting u to a neighbouring intersection v, check whether reaching v through u is shorter than v's current best distance; if so, update v's distance, set v's parent to u, and push the improved (distance, v) pair into the queue.
- Once the queue is empty, compare the finalized distances of all candidate hospitals and select the one with the minimum value.
- Reconstruct the navigable route to the chosen hospital by walking the parent[] links back to the source and reversing them.
- Whenever new traffic data updates a road's travel time, apply the update to the graph and repeat the steps above from the ambulance's current location.

6. Pseudocode

```
procedure DIJKSTRA(network, source)
    for each intersection v in network do
        dist[v] <- infinity
        parent[v] <- NONE
    end for
    dist[source] <- 0
    Q <- empty min-priority queue
    Q.push( (0, source) )

    while Q is not empty do
        (d, u) <- Q.extractMin()
        if d > dist[u] then continue        // stale entry
        for each road (u, v, travelTime) in network do
            if dist[u] + travelTime < dist[v] then
                dist[v] <- dist[u] + travelTime
                parent[v] <- u
                Q.push( (dist[v], v) )
            end if
        end for
    end while
    return (dist, parent)
end procedure

procedure BUILD-PATH(parent, target)
    path <- empty list
    v <- target
    while v != NONE do
        path.prepend(v)
        v <- parent[v]
    end while
    return path
end procedure

procedure NEAREST-HOSPITAL(dist, hospitals)
    best <- NONE; bestTime <- infinity
    for each h in hospitals do
        if dist[h] < bestTime then
            bestTime <- dist[h]; best <- h
        end if
    end for
    return (best, bestTime)
end procedure
```

```
procedure ROUTE-AMBULANCE(network, source, hospitals)
    (dist, parent) <- DIJKSTRA(network, source)
    (best, bestTime) <- NEAREST-HOSPITAL(dist, hospitals)
    route <- BUILD-PATH(parent, best)
    return (best, bestTime, route)
end procedure

// Whenever new traffic data arrives:
//   network.applyTrafficUpdate(u, v, newTravelTime)
//   ROUTE-AMBULANCE(network, ambulanceSource, hospitals)   // simply re-run
```

7. Flow Diagram



8. Worked Example

Consider a small road network of 8 intersections (0-7). The ambulance starts at intersection 0, and hospitals are located at intersections 6 and 7:

8.1 Road Segments

Road (u - v)	Travel Time (min)
0 - 1	4
0 - 2	2
1 - 2	1

1 - 3	7
2 - 3	4
2 - 4	10
3 - 4	2
3 - 5	6
4 - 5	3
4 - 7	8
5 - 6	1
6 - 7	4

8.2 Pass 1 - Normal Traffic: Initialize and Relax in Greedy Order

dist = [0, ∞, ∞, ∞, ∞, ∞, ∞, ∞] (source = 0)

- Extract 0 (dist 0): relax 1 → 4, 2 → 2
- Extract 2 (dist 2): relax 1 → min(4, 3) = 3, 3 → 2+4=6, 4 → 2+10=12
- Extract 1 (dist 3): relax 3 → 3+7=10 (no improvement over 6)
- Extract 3 (dist 6): relax 4 → 6+2=8, 5 → 6+6=12
- Extract 4 (dist 8): relax 5 → 8+3=11, 7 → 8+8=16
- Extract 5 (dist 11): relax 6 → 11+1=12
- Extract 6 (dist 12): relax 7 → 12+4=16 (no improvement)
- Extract 7 (dist 16): no unrelaxed neighbours remain

Final distances: dist = [0, 3, 2, 6, 8, 11, 12, 16]

Hospitals are at intersections 6 (distance 12) and 7 (distance 16).

Nearest hospital → intersection 6, travel time = 12 minutes, route 0 → 2 → 3 → 4 → 5 → 6 (read off the parent[] chain).

8.3 Pass 2 - After a Live Traffic Update

A congestion report arrives and changes two road weights:

Road (u - v)	Travel Time (min)
4 - 7	8 -> 1
5 - 6	1 -> 9

Re-running Dijkstra from intersection 0 on the updated graph gives dist = [0, 3, 2, 6, 8, 11, 13, 9]. Hospital 6 is now 13 minutes away, but hospital 7 has dropped to 9 minutes - closer than hospital 6.

Nearest hospital → intersection 7, travel time = 9 minutes, route 0 → 2 → 3 → 4 → 7.

This is exactly the scenario Section 4.3 describes: the same algorithm, re-run on the updated graph, is enough to redirect the ambulance the moment congestion makes the original choice slower than an alternative.

9. Program (Java)

The program below loads a road network from a CSV of (intersection, intersection, travelTime) rows, runs Dijkstra's algorithm from the ambulance's location to find the nearest of a list of candidate hospitals together with the actual route to it, then applies a second CSV of live traffic-time updates and re-runs the algorithm to show how the recommended hospital - and the path to it - can change as conditions change.

```
import java.io.*;
import java.util.*;

/**
 * Smart Traffic Management for Emergency Vehicles
 * Finds the nearest reachable hospital from an ambulance's current
 * location using Dijkstra's algorithm, re-routes when live traffic
 * updates change road travel times, and reconstructs the actual
 * turn-by-turn path (not just the travel time) for navigation.
 */
public class AmbulanceRouter {

    /** One directed traversal of a two-way road: destination + current travel time. */
    static class Road {
        final int to;
        int travelTime; // minutes; mutable so live traffic updates can be applied in place

        Road(int to, int travelTime) {
            this.to = to;
            this.travelTime = travelTime;
        }
    }

    /** Adjacency-list road network for the city. */
    static class RoadNetwork {
        final int intersections;
        final List<List<Road>> adj;

        RoadNetwork(int intersections) {
            this.intersections = intersections;
            this.adj = new ArrayList<>();
            for (int i = 0; i < intersections; i++) adj.add(new ArrayList<>());
        }

        void addRoad(int u, int v, int travelTime) {
            adj.get(u).add(new Road(v, travelTime));
            adj.get(v).add(new Road(u, travelTime)); // roads are two-way
        }

        /** Loads (u,v,travelTimeMinutes) rows from a CSV, first line is a header. */
        static RoadNetwork fromCsv(String filename, int intersections) throws IOException {
            RoadNetwork network = new RoadNetwork(intersections);
            try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
                String line = br.readLine(); // skip header
                while ((line = br.readLine()) != null) {
                    if (line.isBlank()) continue;
                    String[] p = line.split(",");
                    network.addRoad(Integer.parseInt(p[0].trim()),
                                    Integer.parseInt(p[1].trim()),
                                    Integer.parseInt(p[2].trim()));
                }
            }
            return network;
        }

        /**
         * Applies live congestion data on top of the existing graph: every
         * (u, v, newTravelTime) row overwrites that road's current weight
         * in both directions, modelling traffic that has changed since load.
         */
        void applyTrafficUpdates(String filename) throws IOException {
            File f = new File(filename);
            if (!f.exists()) return; // no updates this run
            try (BufferedReader br = new BufferedReader(new FileReader(f))) {
                String line = br.readLine(); // skip header
                while ((line = br.readLine()) != null) {
                    if (line.isBlank()) continue;
```

```
                String[] p = line.split(",");
                int u = Integer.parseInt(p[0].trim());
                int v = Integer.parseInt(p[1].trim());
                int newW = Integer.parseInt(p[2].trim());
                for (Road r : adj.get(u)) if (r.to == v) r.travelTime = newW;
                for (Road r : adj.get(v)) if (r.to == u) r.travelTime = newW;
            }
        }
    }
}

/** Holds the outcome of one Dijkstra run: distances plus predecessor links for path recovery. */
static class ShortestPaths {
    final int[] dist;
    final int[] parent;

    ShortestPaths(int[] dist, int[] parent) {
        this.dist = dist;
        this.parent = parent;
    }

    /** Reconstructs the intersection-by-intersection route from source to target. */
    List<Integer> pathTo(int target) {
        LinkedList<Integer> path = new LinkedList<>();
        if (dist[target] == Integer.MAX_VALUE) return path; // unreachable
        for (int at = target; at != -1; at = parent[at]) path.addFirst(at);
        return path;
    }
}

/**
 * Dijkstra's algorithm with a binary-heap min-priority queue.
 * Runs in O((V + E) log V) and also records each node's predecessor
 * so the actual navigable route can be reconstructed afterwards.
 */
static ShortestPaths shortestTravelTimes(RoadNetwork network, int source) {
    int n = network.intersections;
    int[] dist = new int[n];
    int[] parent = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(parent, -1);
    dist[source] = 0;

    // {travelTime, intersection}
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
    pq.add(new int[]{0, source});

    while (!pq.isEmpty()) {
        int[] top = pq.poll();
        int d = top[0], u = top[1];
        if (d > dist[u]) continue; // stale entry, already beaten

        for (Road r : network.adj.get(u)) {
            int candidate = dist[u] + r.travelTime;
            if (candidate < dist[r.to]) {
                dist[r.to] = candidate;
                parent[r.to] = u;
                pq.add(new int[]{candidate, r.to});
            }
        }
    }
    return new ShortestPaths(dist, parent);
}

/** Picks the reachable hospital with the smallest finalized travel time. */
static int nearestHospital(ShortestPaths sp, List<Integer> hospitals) {
    int best = -1, bestTime = Integer.MAX_VALUE;
    for (int h : hospitals) {
        if (sp.dist[h] < bestTime) {
            bestTime = sp.dist[h];
            best = h;
        }
    }
    return best;
}

static void report(String label, RoadNetwork network, int source, List<Integer> hospitals) {
    ShortestPaths sp = shortestTravelTimes(network, source);
    System.out.println("\n--- " + label + " ---");
```

```
        for (int h : hospitals) {
            String time = sp.dist[h] == Integer.MAX_VALUE ? "unreachable" : sp.dist[h] + " min";
            System.out.println("Hospital at intersection " + h + ": " + time);
        }
        int best = nearestHospital(sp, hospitals);
        System.out.println("Nearest hospital: intersection " + best
                + ", travel time " + sp.dist[best] + " minutes.");
        System.out.println("Navigation route: " + String.join(" -> ",
                sp.pathTo(best).stream().map(String::valueOf).toArray(String[]::new)));
    }

    public static void main(String[] args) throws IOException {
        int numIntersections = 8;
        RoadNetwork network = RoadNetwork.fromCsv("road_network.csv", numIntersections);

        int ambulanceSource = 0;
        List<Integer> hospitals = List.of(6, 7);

        System.out.println("Road network loaded: " + numIntersections + " intersections.");
        System.out.println("Ambulance current location: intersection " + ambulanceSource);

        int prevBest = nearestHospital(shortestTravelTimes(network, ambulanceSource), hospitals);
        report("Pass 1: normal traffic conditions", network, ambulanceSource, hospitals);

        System.out.println("\n--- Live traffic update applied ---");
        network.applyTrafficUpdates("traffic_update.csv");

        report("Pass 2: after dynamic traffic update", network, ambulanceSource, hospitals);
        int newBest = nearestHospital(shortestTravelTimes(network, ambulanceSource), hospitals);
        if (prevBest != newBest) {
            System.out.println("\nRouting decision changed due to live traffic: now routing to "
                    + "intersection " + newBest + " instead of " + prevBest + ".");
        } else {
            System.out.println("\nRouting decision unchanged despite traffic update.");
        }
    }
}
```

10. Output

The program was run on an 8-intersection network with hospitals at intersections 6 and 7. In the first pass, intersection 6 is the closer hospital; after a live traffic update clears the road leading to intersection 7 and congests the road leading to intersection 6, the algorithm correctly re-routes the ambulance to intersection 7 instead, and reports the new turn-by-turn path:

```
mohit@TYIT-DAA:~/DAA_Assignment4$ java AmbulanceRouter.java
Road network loaded: 8 intersections.
Ambulance current location: intersection 0

--- Pass 1: normal traffic conditions ---
Hospital at intersection 6: 12 min
Hospital at intersection 7: 16 min
Nearest hospital: intersection 6, travel time 12 minutes.
Navigation route: 0 -> 2 -> 3 -> 4 -> 5 -> 6

--- Live traffic update applied ---

--- Pass 2: after dynamic traffic update ---
Hospital at intersection 6: 13 min
Hospital at intersection 7: 9 min
Nearest hospital: intersection 7, travel time 9 minutes.
Navigation route: 0 -> 2 -> 3 -> 4 -> 7

Routing decision changed due to live traffic: now routing to intersection 7 instead of 6.
mohit@TYIT-DAA:~/DAA_Assignment4$
```

11. Conclusion

This assignment shows how Dijkstra's algorithm, run with a min-priority queue over an adjacency-list road network, computes the shortest travel time from an ambulance's location to every intersection in O((V + E) log V) time, letting the nearest of several candidate hospitals be identified with one additional linear scan. Recording each intersection's parent during relaxation costs nothing extra asymptotically but turns a bare travel-time number into an actual, navigable route - directly answering the assignment's requirement for a usable representation of the optimal path. Because each run is independent and inexpensive, the same algorithm can simply be re-run whenever live traffic data updates a road's travel time; as the output demonstrates, this let the recommended hospital, and the route to reach it, correctly switch once congestion made the original choice slower than an alternative. This combination of efficiency, correctness, and responsiveness to changing conditions is exactly what a real-time emergency dispatch system needs.